

RiskDetector 전시회 발표 자료

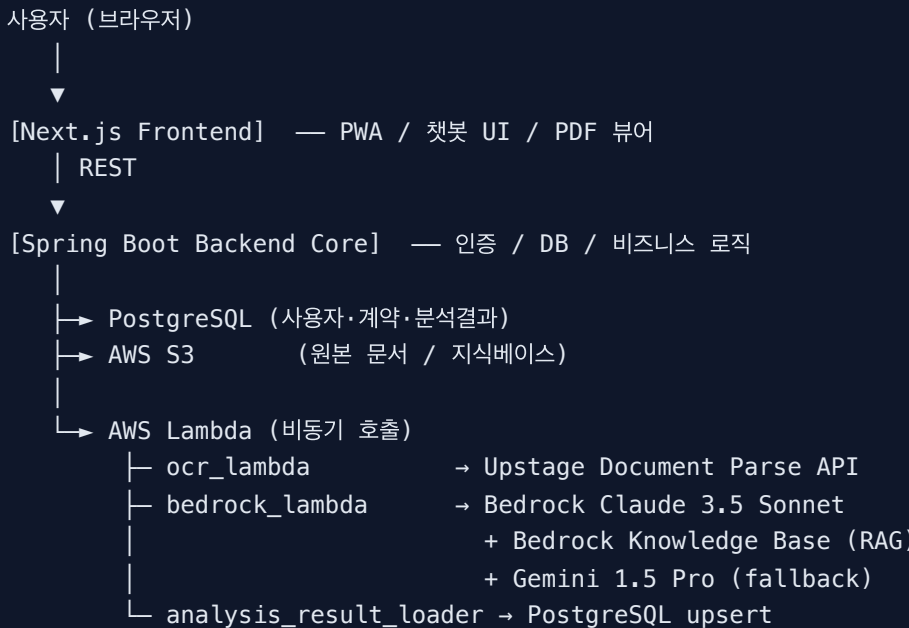
팀 공유용 — 기술 스택 / 선택 이유 / 예상 질문 답변 가이드

1. 프로젝트 한 줄 소개

RiskDetector는 사용자가 업로드한 계약서를 OCR로 디지털화하고, RAG 기반 AI가 한국 법령·판례를 근거로 독소조항을 탐지·설명·수정안까지 제시해주는 웹 서비스다.

핵심 가치: "법률 비전문가도 계약 전에 위험을 알 수 있게 한다"

2. 전체 아키텍처 한눈에



배포: **Render** (Frontend + Backend + Postgres) / **AWS** (Lambda·Bedrock·S3, ap-northeast-2 서울 리전)

3. 기술 스택 + 선택 이유

3.1 Frontend — Next.js 16 + React 19 + TypeScript

기술	버전	역할
Next.js	16.2.2	App Router 기반 SSR/CSR 하이브리드
React	19.2.4	UI 컴포넌트
TypeScript	—	타입 안정성
Tailwind CSS	4	디자인 시스템 / 빠른 스타일링
pdfjs-dist	5.7.x	클라이언트 사이드 PDF 뷰어
Framer Motion	—	자연스러운 인터랙션 / 챗봇 모션
dompurify	—	LLM 출력물 HTML 삭제(sanitize)
OpenAI SDK	6.38.0	챗봇(아르디) 응답 스트리밍

왜 Next.js? - App Router로 인증 보호 라우트(`/analysis` , `/my`)와 공개 라우트(`/`)를 깔끔히 분리. - Server-side rewrites로 모든 `/api/*` 호출을 Spring 백엔드에 프록시 → CORS 이슈 회피, 토큰을 브라우저에 노출 안 함. - PWA 지원: 오프라인에서도 분석 결과 페이지를 다시 볼 수 있음.

왜 Tailwind? - 전시 직전까지 디자인이 계속 바뀌어서 빠른 이터레이션 필요. - 디자인 토큰(`--rd-blue` , `--rd-risk-hi` 등)을 CSS 변수로 분리해 다크/라이트 톤도 한 곳에서 조정.

3.2 Backend Core — Spring Boot 3.5 + Java 17

기술	역할
Spring Boot 3.5.13	REST API 서버
Spring Data JPA	ORM, 도메인 모델
Spring Security + OAuth2 Client	Google 로그인
JJWT 0.12.3	세션리스 JWT 인증
PostgreSQL 15 JDBC	메인 DB
AWS SDK v2	S3 업로드 / Lambda 비동기 호출
jsoup	HTML 파싱 (계약 원문 구조화)

왜 Spring Boot? - 팀 인원 중 Java 백엔드 경험자가 있어 학습 부담이 낮음. - Spring Security + OAuth2 Client 조합으로 Google 로그인 / JWT 발급을 표준 방식으로 구현. - AWS SDK v2와 결합해 S3 / Lambda 호출을 동기·비동기 모두 안전하게 다룰 수 있음.

핵심 도메인: User , Contract , ContractAnalysis , OcrContent , ToxicClause , LegalTip , LegalTipBookmark

주요 엔드포인트: - POST /api/ocr/upload — 문서 업로드 → OCR Lambda 트리거 - POST /api/analysis — 분석 실행 → Bedrock Lambda 트리거 - GET /api/contracts/{id}/analysis/{aid} — 분석 결과 조회 - POST /api/chatbot/retrieve — 챗봇이 사용할 법령 컨텍스트 검색 - GET /api/tips — 법률 팁 피드 - GET /api/auth/me — 현재 로그인 사용자 정보

3.3 Backend AI — AWS Lambda (Python 3.13)

비동기 워커 3개를 Lambda로 분리:

(1) ocr_lambda

- Upstage Document Parse API 호출 (한국어 OCR에 강함)
- PDF/이미지 → 구조화된 텍스트(블록 단위)
- 결과를 S3에 저장 후, 백엔드에 콜백

왜 Upstage? - 한국어 계약서(도장·필기·표 혼합)에 대해 Tesseract, Google Vision보다 정확도가 높았음. - 블록 단위(헤딩/문단)로 구조를 보존해줘서 본문 하이라이트 매칭이 쉬워짐.

(2) bedrock_lambda

- 메인 LLM: AWS Bedrock — Claude 3.5 Sonnet (anthropic.claude-3-5-sonnet-20241022-v2:0)
- RAG: Bedrock Knowledge Base (BM25 + 시맨틱 하이브리드 검색)
- Reranker: 선택적으로 Cohere / Amazon Rerank
- Fallback: Google Gemini 1.5 Pro (Bedrock rate limit / 장애 시 자동 폴백)

왜 Claude + Bedrock? - 한국어 법률 문서 추론 성능이 GPT-4 / Gemini 대비 안정적. - Bedrock에서 운영하면 IAM 기반 권한·VPC·로깅이 일원화되고, Knowledge Base가 통합 제공되어 별도 벡터 DB 운영이 불필요. - 같은 서울 리전(ap-northeast-2)에 두면 레이턴시도 짧음.

왜 Gemini를 fallback으로? - 전사·시연 도중 Bedrock throttling 한 번에 데모가 망가지면 안 되므로, 동일 인터페이스로 폴백 경로를 두 줄로 구현.

(3) analysis_result_loader

- Bedrock Lambda가 분석을 끝내면 직접 PostgreSQL에 결과를 upsert.
- 백엔드와 분리해서 분석 워커가 DB write까지 책임지고, API 서버는 read-only로 결과를 노출.

왜 Lambda 3개로 나눴나? - OCR / 분석 / 저장은 각각 외부 의존성·런타임이 달라 실패 도메인이 다름. - 분리하면 부분 재시도(예: OCR은 성공, 분석만 실패) 처리가 단순해짐. - Spring 백엔드는 응답시간 SLA가 다르므로(즉시 응답), 무

거운 일은 fire-and-forget으로 던지는 게 안정적.

3.4 RAG 파이프라인 — ai_rag

데이터 출처: - 국가법령정보 Open API — 한국 법령 본문 + 판례 256건 큐레이션 - **찾기 쉬운 생활법령(Easylaw)** — 부동산/임대차, 근로/노동 Q&A 600건 - 모두 `s3://risk-detector-myongji/` 에 텍스트 포맷으로 적재

왜 Bedrock Knowledge Base? - 자체 벡터 DB(FAISS, pgvector, Pinecone)를 운영하면 인덱스 갱신·임베딩 모델 관리·인프라 비용이 추가됨. - KB는 S3에 파일만 올려두면 자동으로 청크·임베딩·인덱싱·하이브리드 검색까지 처리. - 6명 팀에서 인프라 운영 부담을 최소화하는 결정.

검색 방식: BM25(키워드) + 시맨틱(임베딩) **하이브리드** — 법률은 정확한 단어(예: "전속", "위약금")가 결정적이라 BM25가, 의도 매칭은 시맨틱이 보완.

3.5 Database / Storage / Infra

- **PostgreSQL 15:** 도메인 데이터 / 분석 결과 / 북마크
- **AWS S3 (ap-northeast-2):** 원본 문서 + 지식베이스 소스
- **Docker Compose:** 로컬 개발 환경 (Postgres + Backend + Frontend)
- **Render:** 프로덕션 호스팅 (백엔드·프론트·DB)
- **AWS:** Lambda + Bedrock + KB + S3

4. 핵심 기능별 기술 흐름

4.1 계약서 업로드 → 분석 완료까지

- 1) 사용자가 PDF 업로드
 - ↳ Frontend가 `/api/ocr/upload` 로 multipart 전송
- 2) Backend Core가 S3에 원본 저장 + `ocr_lambda` 비동기 호출
- 3) `ocr_lambda` → Upstage API → 텍스트 블록 추출 → S3 결과 저장
- 4) Backend가 ocr 완료 콜백 받으면 `bedrock_lambda` 호출
- 5) `bedrock_lambda`
 - 계약 유형 추론 (`entertainment / labor / lease ...`)
 - Knowledge Base 하이브리드 검색으로 관련 법령/판례 retrieval
 - Claude 3.5 Sonnet에 컨텍스트 + 계약 원문 주입
 - 독소조항 리스트 + 위험도 + 이유 + 수정 제안 생성
- 6) `analysis_result_loader` → Postgres upsert
- 7) Frontend가 폴링으로 `status=completed` 확인 후 결과 페이지 진입

4.2 아르디 챗봇

- 사용자가 분석 결과 페이지에서 "아르디에게 물어보기" 클릭
- Frontend가 `/api/chatbot/retrieve` 로 백엔드에 컨텍스트 요청 (선택된 조항 + 분석 메타데이터)
- 백엔드가 Knowledge Base에서 관련 법령 retrieval
- Frontend가 retrieval 결과 + 사용자 질문을 묶어 **OpenAI 스트리밍 호출**
- 답변에 인용된 근거(법령 조문)는 채팅 응답 옆에 카드로 표시

왜 챗봇은 OpenAI인가? - 스트리밍 응답 UX(타이핑 효과)가 Bedrock보다 토큰 단위 적응성이 좋았음. - 챗봇은 "대화"라는 일반 도메인이라 굳이 Claude를 안 써도 충분하고, 비용도 더 저렴. - 분석(중요·정확성 우선) vs 챗봇(즉응·자연스러움 우선) 역할 분리.

5. 예상 질문 & 답변 가이드

A. 서비스 / 제품 관련

Q1. 왜 이걸 만들었나요?

부동산·근로 계약처럼 일상에서 접하는 계약도 조항 하나 잘못 보면 손해가 크다. 변호사 상담은 비용·접근성이 떨어지고, 사람들이 "혹시 이거 이상한 거 아닌가?" 를 가볍게 검증할 수 있는 1차 필터가 필요하다고 봤다.

Q2. 변호사를 대체하나요?

아니다. 분석 결과는 **법률 자문이 아닌 참고 자료**로 명시한다. 사용자가 위험 신호를 먼저 인지하고, 중요한 결정은 전문가 상담을 받도록 UI에서도 안내한다.

Q3. 어떤 계약서를 지원하나요?

현재는 **부동산 임대차 / 근로 / 엔터테인먼트(전속)** 계약에 최적화. KB에 이 도메인 법령·판례·Q&A가 적재되어 있다. 다른 유형도 일반 분석은 동작하지만 정확도는 위 3개가 높다.

Q4. 비즈니스 모델은?

MVP는 무료 + 로그인 사용자에게 분석 이력 / 챗봇 추가 기능 제공. 향후 변호사 상담 매칭이나 계약서 템플릿마켓 연계 가능.

B. 기술 / 정확도 관련

Q5. AI가 틀린 분석을 내면 어떻게 하나요?

세 가지 안전장치를 둔다. 1. **RAG로 근거 의무화** — Claude는 KB에서 retrieve된 법령 텍스트를 인용하도록 프롬프트가 강제. 2. **위험도 3단계** — "주의/확인 권장/안전"으로 표시해 단정적 판단을 피함. 3. **항상 면책 안내** — 모든 분석 결과에 "법률 자문 아님" 문구 노출.

Q6. OCR 정확도는?

Upstage Document Parse를 쓴다. 한국어 계약서(도장·표·필기) 혼합 케이스에서 Tesseract, Google Vision 대비 정확도와 구조 보존이 더 좋았다.

Q7. 분석에 시간이 얼마나 걸리나요?

OCR 평균 10~20초 + 분석(Bedrock) 평균 30~60초. 총 1분 안팎. 로딩 페이지에 아르디의 팁을 보여줘 체감 대기 시간을 줄였다.

Q8. 왜 Claude(Bedrock)인가요? GPT-4도 있는데?

1) 한국어 법률 텍스트 long-context 추론에서 Claude 3.5 Sonnet이 안정적. 2) Bedrock에서 운영하면 KB·IAM·로깅이 통합되어 운영 비용이 낮다. 3) 챗봇용으로는 OpenAI도 병행. 도메인별 역할 분리.

Q9. 자체 벡터DB가 아니라 Bedrock KB를 쓴 이유?

6명 팀에서 인덱스 관리·임베딩 모델 업데이트·서버 운영을 따로 하지 않으려고. S3에 파일만 올리면 KB가 청크·임베딩·하이브리드 검색까지 처리. 데모 안정성 우선.

Q10. RAG가 가져오는 데이터는 어디서?

국가법령정보 Open API(법령 + 판례 256건)와 찾기쉬운 생활법령(Q&A 600건). 모두 공개 자료. 크롤링 코드는 `ai_rag/` 에 있고, S3에 정형 텍스트로 적재한다.

Q11. 왜 검색이 BM25 + 시맨틱 하이브리드인가요?

법률은 단어가 결정적(예: "전속" vs "독점"은 의미가 다름). BM25가 정확 매칭을, 시맨틱이 의도 매칭을 잡아준다. KB가 이걸 한 번에 처리.

Q12. Lambda를 왜 3개로 쪼갰나요?

OCR / 분석 / DB 적재는 외부 의존성과 실패 양상이 다르다. 분리해두면 부분 재시도, 모니터링, 비용 추적이 단순해진다. 또한 Spring 백엔드는 즉시 응답해야 해서 무거운 일은 fire-and-forget으로 던진다.

Q13. Gemini fallback은 왜?

전시·시연 도중 Bedrock 한쪽이 throttling 되면 데모가 멈춘다. 동일 인터페이스로 Gemini 1.5 Pro 폴백 경로를 두 줄로 추가해 둬.

C. 보안 / 개인정보

Q14. 업로드한 계약서는 어디 저장되나요?

서울 리전(ap-northeast-2) S3에 저장된다. 비로그인(게스트)도 사용할 수 있고, 게스트 ID는 브라우저 로컬에 보관된다. 추후 자동 삭제 정책을 추가할 계획.

Q15. 인증은 어떻게?

Google OAuth2 + JWT. 백엔드에서 OAuth 콜백을 처리해 JWT를 HttpOnly 쿠키로 내려준다. 게스트는 `X-Guest-Id` 헤더로 식별.

Q16. LLM에 계약서 내용이 학습되지 않나요?

Bedrock·OpenAI 모두 기본적으로 API 호출 데이터는 학습에 사용되지 않는 옵션. 우리는 그 모드로 호출한다.

D. 운영 / 확장성

Q17. 어디에 배포되나요?

Frontend·Backend·DB는 Render, AI 워커와 KB는 AWS(ap-northeast-2). Docker Compose로 로컬 개발도 그대로 구동된다.

Q18. 사용자가 갑자기 늘면?

Lambda는 자동 스케일링. Backend Core(Spring)는 Render 인스턴스를 늘리면 되고, 분석은 결국 Bedrock TPS가 병목이라 그 시점에 Provisioned Throughput을 사면 된다.

Q19. 모바일 지원은?

PWA로 설치 가능. 반응형 레이아웃과 모바일 전용 바텀시트(분석 컨텍스트), 하단 네비게이션을 별도로 구현.

Q20. 비용은 대략?

메인 비용은 Bedrock(분석 호출당 토큰 비용) + KB(저장·쿼리). 시연 수준에서는 월 수만 원 단위. 트래픽이 늘면 OpenAI 챗봇 비용이 가장 빠르게 증가.

E. 디자인 / UX

Q21. 분석 결과 페이지의 핵심 UX 포인트는?

좌측 본문에 위험 조항을 **인라인 하이라이트**로 표시 → 클릭하면 우측 패널이 같은 조항의 위험 이유 ·BEFORE/AFTER·근거 법령을 보여줌. 본문이 길어도 우측 패널은 sticky로 따라오며 자체 스크롤된다.

Q22. 본문에서 위치를 못 찾은 위험 조항은?

본문 카드 하단에 "본문에서 위치를 찾지 못한 위험 조항" 섹션으로 별도 노출. 클릭하면 동일하게 우측 상세가 열린다.

Q23. 아르디(챗봇) 캐릭터의 의도는?

법률은 위압감이 큰 도메인이라, 친근한 마스코트를 두어 "물어봐도 되는 분위기"를 만들고 싶었다. 응답 옆에 인용 근거를 카드로 붙여 신뢰감도 보강.

6. 데모 시 체크리스트 (발표 직전)

- [] Render 백엔드 / 프론트 모두 깨어 있는지 (cold start 30초 정도 걸림 — 미리 한번 깨워두기)
- [] Bedrock 키 / OpenAI 키 만료 안 됐는지
- [] 시연용 샘플 계약서 1~2개 미리 업로드 → 분석 완료 상태로 캐시
- [] 챗봇 warmup 호출 한번 (`/api/chatbot/warmup`)
- [] 모바일 PWA 설치 데모도 준비 (휴대폰 한 대)
- [] 네트워크 끊겼을 때 폴백 시나리오: 이미 분석 완료된 결과 페이지가 PWA 캐시로 열리는 것 보여주기

7. 한 줄 요약 (질문 받기 전 인트로용)

"RiskDetector는 **한국어 법률 RAG**로 학습된 Claude가, 사용자의 계약서에서 **독소조항을 찾아 근거 법령과 함께 설명·수정안을 제시**해주는 서비스입니다. Next.js·Spring Boot·AWS Bedrock·Lambda로 구성되어 있고, 분석 정확도와 운영 안정성을 위해 **RAG + LLM 폴백 + 비동기 워커 분리** 구조를 택했습니다."